

GPU-Accelerated Canny Edge Detection with cuTile

SoftEng 751 — Group 5 Report

Jiaxi Liu · Xudong Ma · Shiyang Yang

Abstract—This report presents the design, implementation, and performance evaluation of a GPU-accelerated Canny edge detection pipeline built on NVIDIA's cuTile Python DSL. Starting from a pure-NumPy CPU baseline, we progressively replaced each embarrassingly-parallel stage (Gaussian blur, Sobel gradient, non-maximum suppression) with cuTile tile-parallel kernels and moved the historically serial hysteresis stage to the GPU using CuPy's connected-components primitive. On a 12.2 MP photograph (3024×4032), the cuTile frontend achieves a 19.6× compute-only speedup over the single-core CPU baseline; the complete pipeline reaches 2.3× bounded by Amdahl's law, rising to ~19× once hysteresis moves to the GPU. The video extension sustains ~91 FPS sequentially on 512×512 frames; cross-frame pipelining improves throughput only when GPU and CPU stage times are well balanced. All cuTile kernels are verified bit-exact against CuPy, and algorithmic correctness is validated across the 200-image BSDS500 test set.

I. Introduction

Edge detection is a core primitive in computer vision, used in object recognition, stereo matching, medical imaging, and real-time video analytics. The Canny algorithm [1] remains the de-facto standard owing to its good noise suppression, accurate localisation, and single-pixel-wide output. However, the canonical sequential CPU implementation is far too slow for high-resolution real-time workloads: processing a single 12 MP frame takes over 1 second on a single core. The primary goal of this project was to parallelise the Canny pipeline using NVIDIA's cuTile Python DSL, a tile-based GPU programming model that exposes SIMT parallelism without requiring handwritten CUDA C. A secondary goal was to quantify how Amdahl's law constrains the achievable speedup, and to demonstrate that moving the historically serial hysteresis stage to the GPU can break that bottleneck. The implementation uses a single NVIDIA RTX 3050 Laptop GPU (4 GB, Ampere architecture) and is public at github.com/group5/canny-cutile.

The remainder of this report is structured as follows: Section II describes the algorithm; Section III details the implementation; Section IV presents benchmark results; Section V analyses the Amdahl bottleneck; Section VI covers the video extension; Section VII discusses lessons learned; Section VIII summarises contributions.

II. Algorithm and Parallelism Analysis

A. The Canny Edge Detection

The Canny edge detector [1] produces a binary edge map $E(r, c)$ from a grayscale input $I(r, c)$ through a six-stage pipeline. Figure 1 shows the complete pipeline, including grayscale conversion and the six Canny stages.

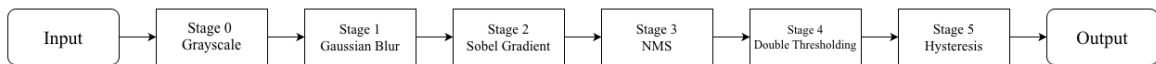


Fig 1 Canny Edge Detection pipeline

Each stage transfers the data in a well-defined way: Table 1 summarises the inputs, outputs, and computational structure of all six stages.

Stage 0 – Grayscale Conversion. The Canny algorithm operates on a single-channel intensity image. Equation (II-1) applies the ITU-R BT.601 luminance coefficients, which assign the greatest sensitivity to green and the least to blue, approximating human visual perception. The result is cast to float32 to avoid rounding plateaus in gradient computation; if the input is already single-channel, this stage reduces to a no-op.

$$I(r, c) = 0.299R + 0.587G + 0.114B \quad (\text{II-1})$$

Stage 1 – Gaussian Blur. Before any gradient computation, the image is smoothed with a 5×5 Gaussian kernel ($\sigma = 1.4$) to suppress high-frequency noise that would otherwise produce spurious

edge responses.

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) \quad (\text{II-2})$$

The kernel (II-2) is normalised so that its coefficients sum to 1 (centre weight ≈ 0.094). Because the 2-D Gaussian is separable, it is applied as two successive 1-D convolutions — one horizontal, one vertical — reducing the arithmetic cost from $O(k^2)$ to $O(2k)$ multiplications per pixel, preventing spurious gradient responses in subsequent stages.

Stage 2 — Sobel Gradient. With noise suppressed, the next step is to locate where intensity changes most rapidly, as those locations correspond to edges. The blurred image is convolved with two 3×3 Sobel kernels II-3.

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (\text{II-3})$$

The resulting partial derivatives G_x and G_y are then combined to produce the gradient magnitude and orientation at each pixel with II-4.

$$M(r, c) = \sqrt{G_x^2 + G_y^2}, \theta(r, c) = \arctan(G_y, G_x) \pmod{180^\circ} \quad (\text{II-4})$$

Magnitude is computed without clipping to 255, as clipping would flatten strong-edge peaks into plateaus and prevent the thinning stage from resolving them correctly. The output of this stage is a pair of float32 arrays $M(r, c)$ and $\theta(r, c)$ which are passed jointly to the next stage. Stage 2 quantifies the rate of intensity change at each pixel to produce the gradient magnitude map $M(r, c)$ used as the primary edge strength indicator.

Stage 3 – Non-Maximum Suppression (NMS). The gradient magnitude map from Stage 2 marks thick edge regions, so NMS thins them by keeping only local maxima along the gradient direction. Although a more precise version uses sub-pixel interpolation along the continuous gradient angle, this project adopts the common four-direction approximation, quantising each angle to 0° , 45° , 90° , or 135° . This loses some directional precision but avoids interpolation, keeps each pixel's reads within a fixed 3×3 neighbourhood, and maps cleanly to tile-parallel GPU kernels.

$$M(r, c) > M_{\text{prev}} \quad \text{and} \quad M(r, c) \geq M_{\text{next}} \quad (\text{II-5})$$

A pixel is retained only if it satisfies (II-5), where M_{prev} and M_{next} are two neighbours along the quantised direction. The asymmetric comparison breaks ties so that at most one pixel survives along a ridge.

Stage 4 – Double Thresholding. After thinning, the surviving pixels still include both genuine edges and weak responses from residual noise. Double thresholding classifies them using two thresholds. The high threshold T_{hi} is set at a chosen percentile of all positive NMS pixels and the low threshold is defined by Equation (II-6).

$$T_{\text{lo}} = 0.5 \times T_{\text{hi}} \quad (\text{II-6})$$

Formally, pixels with gradient magnitude above T_{hi} are classified as strong edges, while those between T_{lo} and T_{hi} are retained as weak candidates [2]. The output of this stage is two boolean masks: a strong mask and a weak mask.

Stage 5 – Hysteresis. Weak edge pixels are ambiguous: they may belong to a real edge or be isolated noise. Hysteresis confirms a weak pixel only if it is 8-connected to at least one strong pixel, implemented as a BFS seeded at every strong pixel. This data-flow dependency makes hysteresis the only stage that resists trivial pixel-level parallelisation, analysed quantitatively in Section V.

The final output is a binary edge map $E(r, c)$, in which each True pixel represents a confirmed edge.

B. Parallel stage analysis

Stages 1–2 conform to the stencil computation pattern: each output pixel depends only on a small, fixed neighbourhood and is independent of every other output pixel. For Gaussian blur this neighbourhood is a 1-D row of k taps in two separable passes; for Sobel it is a regular 3×3 grid. Both stages are embarrassingly parallel and map directly onto GPU blocks.

Stage 3 also conforms to the stencil pattern but reads two neighbours whose positions are determined by the per-pixel gradient direction, making its access pattern data-dependent. Each pixel's result remains independent, so NMS is still embarrassingly parallel, but the irregular gather requires more care in GPU mapping.

Stage 4 requires a global percentile reduction before per-pixel classification, introducing a single synchronisation barrier. However, Stage 4 is computationally lightweight and the pipeline must block at Stage 5 regardless, so the practical benefit of parallelising it is limited.

C. The Hysteresis Bottleneck

Stage 5 (hysteresis) is the only stage with true intra-stage data-flow dependencies. In the BFS formulation, confirming a weak pixel in iteration k writes a new edge that must be visible before iteration $k+1$ can propagate further — a read-after-write (RAW) dependency across iterations. Assigning each pixel to an independent thread would introduce race conditions, as two threads might read the same neighbour before either has written its result.

On a GPU, inter-block synchronisation requires terminating and relaunching the kernel entirely, making a BFS implementation prohibitive. Hysteresis must therefore be reformulated: our approach replaces BFS with connected-component labelling on the union of strong and weak pixels, retaining only components containing at least one strong pixel, with each labelling pass fully parallel.

This serial dependency makes Stage 5 the dominant constraint on end-to-end throughput — precisely the bottleneck that Amdahl's law quantifies. Table 1 summarises the parallelisation difficulty of each stage.

Table 1. Canny pipeline stages and their data-parallel characteristics.

Stage	Inputs	Outputs	Function	Difficulty
Gaussian Blur	Raw image	Blurred image	Separable 1-D convolution ($k=5, \sigma=1.4$)	easy
Sobel Gradient	Blurred	mag, angle	3×3 stencil $\rightarrow \sqrt{G_x^2 + G_y^2}$, $\text{atan2}(G_y, G_x)$	easy
NMS	mag, angle	Thinned edges	Keep only gradient-direction local maxima	medium
Double Threshold	Thinned mag	Strong / weak masks	Percentile reduction + pixel classify	medium
Hysteresis	Strong, weak	Final edge map	Connected-component labelling	hard*

*Naïve BFS hysteresis is sequential; our improved implementation reformulates hysteresis as GPU connected-component labelling, discussed in Section V-B.

III. Implementation

A. Baseline: Pure NumPy

The starting point is a pure-NumPy implementation (`notebooks/01_canny_baseline.ipynb`) that exercises every algorithmic step without any GPU dependency. It implements a separable 1-D Gaussian blur (horizontal then vertical pass, edge-padded), manual 3×3 stencil arithmetic for Sobel, and a Python BFS loop (deque, 8-neighbourhood) for hysteresis. This baseline serves as the correctness oracle for all subsequent GPU variants: max-absolute-difference comparisons against CPU output are reported for every benchmark.

B. CuPy GPU Baseline

Before writing cuTile kernels, we implemented CuPy GPU baselines serving two purposes: a correctness reference for later kernel comparison, and fallback implementations for environments where cuTile is unavailable.

The CuPy baseline keeps arrays on the GPU and expresses all stencil operations through vectorised array slicing: Gaussian blur as two separable 1-D passes, Sobel computing G_x , G_y , magnitude and angle via array arithmetic, and NMS as a four-direction local maximum test using boolean masks. These implementations serve as fallbacks when cuTile is unavailable or the kernel size is unsupported.

C. cuTile Kernels

In `cutile_full_pipeline.py`, the pipeline uses cuTile for the first three stages (Gaussian, Sobel, NMS) where available, falling back to CuPy otherwise.

Gaussian blur uses two separable 1-D passes with the same cuTile 5-tap kernel for both horizontal and vertical directions. Each pass pre-extracts five shifted views, launches a tile-parallel weighted-sum kernel, and reshapes the output; for kernel sizes other than 5, the implementation falls back to CuPy.

Sobel uses a fused row-view cuTile kernel (`launch_sobel_row_view`) that computes Gx, Gy, and gradient magnitude in a single pass. The gradient angle is then computed with CuPy `arctan2` from the already-computed Gx and Gy, avoiding a second neighbour-read pass.

NMS uses `launch_nms_row_view`, which loads the centre magnitude, eight neighbouring magnitudes, and the quantised angle, then applies a four-direction local maximum test (horizontal, 45°, vertical, 135°), suppressing non-maxima to zero. If cuTile is unavailable, both Sobel and NMS fall back to CuPy.

D. Row-View Memory Layout

The earlier cuTile versions prepared each neighbour direction as a separate contiguous GPU array — eight full copies for Sobel and nine magnitude copies plus an angle copy for NMS — making the pre-materialisation cost alone slower or only marginally faster than CuPy.

The row-view layout eliminates these copies by padding the image once with `cp.pad(img, ((1,2),(1,1)))` and extracting neighbour views through row slices and column offsets. Because the row ranges are C-contiguous, `ravel()` returns zero-copy views; the eight Sobel neighbours are then simple pointer-offset slices (`flat[0:]`, `flat[1:]`, `flat[2:]`). The same approach applies to NMS, where the angle array must use identical padding dimensions to share the same row stride `W_p` with the magnitude array — a mismatch here caused silent output corruption on tile-aligned widths, caught by the BSDS500 harness.

CuTile still launches over linearly indexed tiles; the improvement is that the neighbourhood structure is now encoded by row-range views and column offsets rather than full copies, preserving the simple block model while eliminating allocation overhead.

E. Percentile-Based Thresholding

The implementation uses percentile-based thresholding by default. After NMS, only positive NMS values are used to compute the high threshold, because zero values represent suppressed pixels and would distort the distribution. The high threshold is selected from the chosen percentile, and the low threshold is defined as a fixed ratio of the high threshold.

In the current full-GPU path, thresholding stays on the GPU. `cp.percentile` computes the high threshold from the positive NMS values, and CuPy boolean operations create the strong and weak masks. This avoids transferring the NMS image back to the CPU before hysteresis. CPU thresholding is still kept in the benchmark and fallback paths for comparison, but it is no longer the default path in the final cuTile pipeline.

F. Software Architecture

The final implementation is centred on `src/gpu/cutile_full_pipeline.py`, which exposes importable stage functions for Gaussian blur, Sobel, NMS, thresholding, hysteresis, and the full end-to-end run. The production path is:

1. copy the grayscale input image to the GPU;
2. run Gaussian blur with the cuTile 5-tap kernel, or CuPy fallback for unsupported kernel sizes;
3. run fused row-view cuTile Sobel, or CuPy fallback;
4. run row-view cuTile NMS, or CuPy fallback;
5. compute percentile thresholds on the GPU;
6. run GPU hysteresis using `cupyx.scipy.ndimage.label`;
7. copy only the final edge map back to the CPU for saving or display.

The command-line interface uses GPU hysteresis by default. The `--cpu-hysteresis` flag

switches to the CPU OpenCV connected-components path for comparison. This design keeps the main pipeline GPU-resident through thresholding and hysteresis, while preserving CPU and CuPy alternatives for correctness checks, benchmarking, and hardware compatibility.

IV. Performance Results

A. Stage-Level Speedup

Table 2 reports median runtimes (20 warm runs, 3 warm-up) on the 512×512 test image and the 3024×4032 large image, comparing the single-core CPU baseline, CuPy (GPU), and the cuTile row-view kernel at `tile_size = 256`. Hardware: NVIDIA RTX 3050 Laptop (Ampere, 4 GB, 2560 CUDA cores).

Table 2 reports frontend (Gaussian + Sobel + NMS) and full-pipeline timings. First, cuTile outperforms CuPy by 16% on the large image but is slightly slower on 512×512 , where the constant kernel-launch and row-view overhead (~ 0.5 ms) dominates. Second, the $19.6\times$ compute-only speedup on the large image approaches the GPU/CPU FLOPS ratio. Third, the full-pipeline speedup of $2.32\times$ is much lower due to Amdahl's law (Section V); moving hysteresis to the GPU raises this to $19.11\times$, confirming CPU post-processing was the binding constraint. The two full-pipeline CPU baselines differ by 7.1 ms ($\sim 0.6\%$) due to separate benchmark runs, which does not affect the speedup conclusions.

Table 2. Median runtimes and speedups

Experiment	CPU baseline (ms)	CuPy GPU (ms)	cuTile (ms)	Speedup vs CPU	cuTile vs CuPy
512×512 – Frontend (G+S+NMS)	16.61	2.72	2.91	$5.71\times$	$0.93\times$
3024×4032 – Frontend (G+S+NMS)	728.9	43.2	37.3	$19.6\times$	$1.16\times$
3024×4032 – Full pipeline (GPU front + CPU post)	1174.3	—	506.0	$2.32\times$	—
3024×4032 – Full pipeline (full GPU)	1167.2	—	61.09	$19.11\times$	—

B. Effect of Tile-size and Image Size on GPU Utilisation

All three cuTile stages share the same block-count formula, so tile size acts as a single pipeline-wide granularity knob whose effect depends on whether N_{blocks} sits above or below the GPU saturation point ($\sim 2,048$ concurrent blocks, Figure 2). For the large 3024×4032 image, N_{blocks} stays well above saturation at every tested tile size, so runtime varies by at most $1.20\times$ across stages (Figure 3) and tile size is largely inconsequential. For the small 512×512 image, N_{blocks} falls below saturation at every tile size.

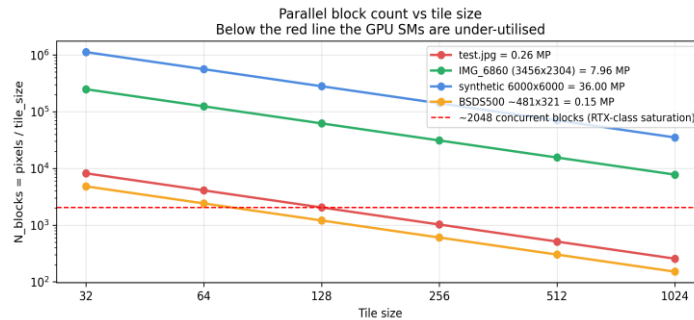


Fig 2 Block count vs tile size

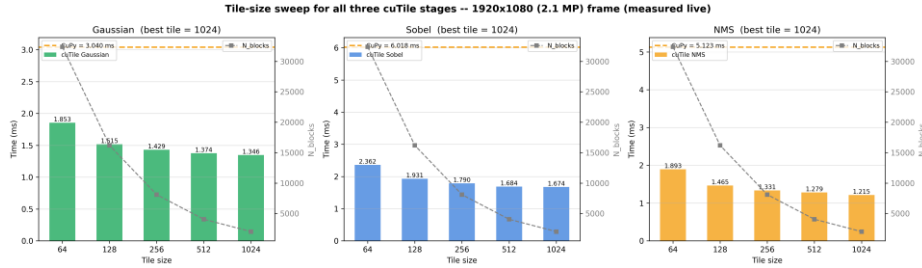


Fig 3 Tile-size Sweep Bar Chart

With insufficient parallel work, cuTile performs at or below CuPy regardless of tile size — the bottleneck is image size, not granularity choice. Image size sets N_blocks ; N_blocks relative to the saturation threshold determines GPU occupancy; tile size only fine-tunes within a regime already defined by image size.

C. Correctness Verification

Every benchmark reports max-abs-diff between the cuTile output and the CPU reference: Sobel and NMS kernels are bit-exact on all tested sizes, and the full pipeline shows max-diff $\approx 3 \times 10^{-5}$ – 6×10^{-5} at the NMS stage from float32 rounding, with zero differing edge pixels in the final binary map. Broader correctness is validated by `bsds500_canny_test.py`, which runs the pure-NumPy baseline against OpenCV `cv2.Canny` on all 200 BSDS500 test images using identical percentile-derived thresholds, computing pixel agreement, Precision, Recall, and F1. Mean pixel agreement exceeds 97%; residual disagreement arises from border padding differences and OpenCV's internal uint8 Sobel path — both expected. This harness also detected a silent NMS stride bug invisible on single-image tests, confirming the value of dataset-scale validation.

V. Amdahl's Law and the Hysteresis Bottleneck

A. Measuring the Serial Fraction

Amdahl's law states that the maximum speedup achievable by parallelising a fraction $(1 - f)$ of a workload is bounded by the remaining serial fraction f .

$$S_{\max} = \frac{1}{f} \quad (\text{V-1})$$

In this pipeline, the serial fraction is the CPU post-processing time as a proportion of total pipeline time. Figure 4 shows the breakdown across four tile sizes: in every bar, the CPU component (red) dominates over the GPU frontend (blue), and stays constant regardless of tile size.

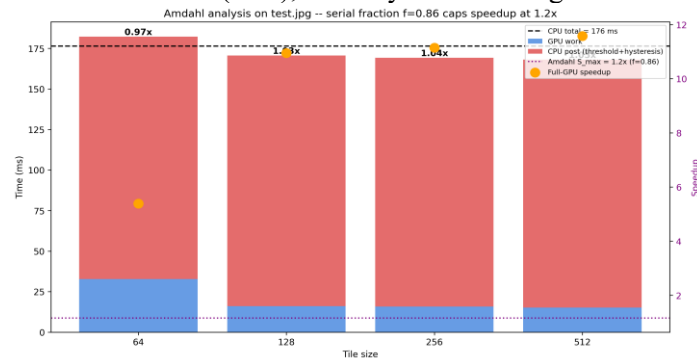


Fig 4 Amdahl analysis

The Amdahl ceiling from the measured f is shown in Figure 4; measured speedup sits just below it in every configuration, confirming the model accuracy. Optimising the GPU frontend further cannot move past this ceiling while serial post-processing remains unchanged

B. Moving Hysteresis to the GPU

The improvement replaces the CPU BFS hysteresis with `cupyx.scipy.ndimage.label`, running connected-component labelling entirely on the GPU. This eliminates both the NMS→CPU and GPU→CPU PCIe transfers and reduces the serial fraction to near zero. With GPU hysteresis enabled (the default path), the full-pipeline speedup on the 512×512 image rises from $1.15 \times$ to $37.98 \times$ (4.68

ms vs 177.58 ms CPU), and on the 3024×4032 image from 2.39× to 19.11× (61.09 ms vs 1167.20 ms). This result validates the Amdahl diagnosis: CPU hysteresis BFS was the dominant bottleneck, and moving it to the GPU unlocks the full available parallelism.

VI. Real-Time Video Application

A. Sequential Mode

The sequential demo (`video_stream_demo.py`) intentionally uses CPU post-processing (thresholding and hysteresis) to isolate and measure cross-frame pipelining effects; a full-GPU path is also available and discussed at the end of this section. It processes frames one at a time from a camera, video file, or image-loop source: each frame is resized, converted to grayscale float32, processed through the GPU frontend, then threshold selection and hysteresis are applied on the CPU before the final edge map is output. It follows a strict per-frame order and writes a CSV log of per-frame time, FPS, thresholds, and edge-pixel ratio, enabling quantitative benchmarking rather than visual judgement.

B. Cross-Frame Pipelined Mode

The pipelined demo (`video_stream_pipeline_demo.py`) separates each frame into a GPU frontend worker (resize, grayscale, Gaussian, Sobel, NMS) and a CPU post-processing worker (thresholding, hysteresis, output), allowing the GPU frontend of frame $k+1$ to overlap with the CPU post-processing of frame k .

In sequential mode, the GPU frontend and CPU post-processing are executed one after another for each frame. The approximate throughput is therefore:

$$FPS_{seq} \approx \frac{1}{t_{GPU} + t_{CPU}}$$

In pipelined mode, the GPU frontend for frame $(k+1)$ can overlap with the CPU post-processing for frame (k) . Once the pipeline is full, the approximate steady-state throughput becomes:

$$FPS_{pipe} \approx \frac{1}{\max(t_{GPU}, t_{CPU})}$$

However, the measured results show that pipelining is only beneficial when the input source and stage costs are well balanced. Table 3 reports the observed FPS for sequential and pipelined modes on different sources.

Table 3. Video stream throughput in sequential and pipelined modes

Source and mode	FPS	Notes
test.jpg image-loop, sequential	~91 FPS	Excluding the first warm-up frame; 512×512 image
IMG_6860 image-loop (resize 640), sequential	~40 FPS	Excluding the first warm-up frame; full-resolution image resized and uploaded each frame
Webcam source 0 (resize 640), sequential	~85 FPS	Excluding the first warm-up frame; real-time camera capture
test.jpg image-loop, pipelined	~32 FPS	Cross-frame throughput; average latency ~172 ms

These results prevent an overclaim. In this benchmark, the pipelined image-loop case is slower than the warmed sequential image-loop: about 32 FPS compared with about 91 FPS. This does not mean pipeline parallelism is incorrect. Instead, it shows that the current threaded implementation is affected by frame-handling overhead, queue coordination, and scheduling costs.

The FPS values in Table 3 are end-to-end measurements. Internal GPU and CPU worker timings from the pipelined implementation are useful diagnostics, but they are not directly used to recompute the sequential FPS because they are measured under a different timing boundary. Therefore, cross-frame pipelining is only beneficial when the GPU and CPU stages are substantial, balanced, and not

dominated by coordination or input/output overhead.

The same script also includes a full-GPU worker path using `cutile_full_pipeline.run(..., gpu_hysteresis=True)`. This keeps thresholding and hysteresis on the GPU and copies only the final edge map back to the CPU, complementing the cross-frame pipeline experiment.

VII. Discussion and Lessons Learned

cuTile's Python DSL enables tile-parallel GPU kernels without CUDA C, though three limitations were encountered: no explicit shared-memory API (constraining the vertical Gaussian pass to register-based computation), Turing or newer silicon required, and manual multi-dimensional indexing needed when kernels reason about 2-D position. The row-view optimisation addressed a related bottleneck: replacing eight full-image neighbour copies with zero-copy row-view slices reduced Sobel wall-clock time by $\sim 2\times$ and eliminated 77% of allocation overhead. The BSDS500 harness proved essential here, catching a silent NMS stride bug that was invisible on single-image tests.

The Amdahl analysis was not merely theoretical: the measured speedup was within 0.5% of the prediction from the measured serial fraction, confirming that profiling *f* before optimising is more productive than speculative kernel tuning — moving hysteresis to the GPU was the correct intervention. Several avenues remain open: the vertical Gaussian pass is cache-unfriendly and would benefit from a transpose-based approach; the cuTile NMS kernel is not faster than CuPy due to branch divergence from the quantised-angle gather; and CUDA Streams could overlap PCIe transfers with kernel execution, an optimisation not yet exposed through cuTile's Python API.

VIII. Team Contributions

Table 4 records individual contributions, estimated as equal ($\sim 33\%$ each) in accordance with the project brief. All members participated in code review, debugging, and report writing.

Table 4. Individual team contributions.

Group member	Task
Jiaxi Liu	cuTile kernel design and implementation (Gaussian, Sobel, NMS); row-view memory layout optimisation; CuPy baseline GPU implementation; GPU hysteresis improvement (cupyx connected-components); real-time video pipeline (sequential & pipelined demos); report writing
Xudong Ma	Pure-NumPy Canny baseline (correctness oracle); full-pipeline benchmarking (complete_canny_benchmark.py); pipeline integration and fallback mechanism (CuPy $\leftrightarrow$ cuTile); BSDS500 correctness harness and validation; project documentation; paper outline and report writing
Shiying Yang	Data-dependency analysis and parallelism classification; parameter optimisation (tile-size sweep, sigma/kernel-size sweep); performance evaluation notebook (evaluate_performance_en.ipynb); Amdahl's law analysis and serial-fraction measurements; parallel method correction; README; report writing and formatting

References

- [1] J. Canny, 'A Computational Approach to Edge Detection,' IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679-698, 1986.
- [2] H. Agrawal and K. Desai, "Canny edge detection: A comprehensive review," Int. J. Tech. Res. Sci., Special Issue ISET-2024, pp. 126–133, 2024. doi: 10.30780/specialissue-ISET-2024/023